IDENTIFIERS FOR DIFFERENT ORDERS IN OMS

PROBLEM STATEMENT: Order cases:

- ***Store Pickup***
- ***Sand sale***
- ***Online order***
- ***POS completed***
- ***MixedCart Orders***
- ***Identifier in (oms and shopify)***
- ***OrderItemAttribute, orderAttribute, OrderItemShipGroup. (mapping with Shopify JSON).***

StorePickup:

An order item is automatically identified and treated as a Store Pickup if it meets any of the following conditions during the Shopify import:

1. The "pickupstore" Keyword:
   The item has a custom attribute (property) where the key contains the word "pickupstore".

2. The Custom Admin Keyword:
The item has a custom attribute that exactly matches a specific keyword defined by the system administrator in the OMS configuration/settings.

3. The Order Tag + Store Assignment:
The overall order contains a specific pre-configured tag (such as "bopis" or "pickup"), and the item includes an attribute indicating the specific store location assigned for fulfillment.

What happens when one of these conditions is true?

If any of the above identifiers are detected, the OMS automatically performs the following actions:

• Changes the shipment method of the item to "STOREPICKUP".

• Removes the shipping carrier assignment, meaning no FedEx and UPS shipment labels will be generated.

• Reads the value of the matching attribute (for example, "29") and assigns the order item to that specific physical store's fulfillment queue/facility.

HIGH LEVEL CODE FLOW

Get Configurations

↓

Loop Through Each Line Item

↓

Normalize Custom Attributes

↓

Check Pre-Selected Facility Logic

↓

If Not Found → Check Generic Pickup Properties

↓

Mark Item as STOREPICKUP

↓

Assign Facility + Shipment Method + Carrier

JSON Payload Location	Extracted JSON Value	Groovy Script Logic / Variable	Effect on the OMS Order	OMS Entity & Field
`lineItems[].customAttributes`	`"key": "_pickupstore"`	`propName.contains("pickupstore")`	Identifies the keyword, triggers the BOPIS logic block, and flags `pickupStore = true`.	*(Internal script boolean, not directly stored)*

`lineItems[].customAttributes`	`"value": "29"`	`fromFacilityId = propValue`	Sets the physical fulfillment facility ID to 29 (Irvine Spectrum).	**Entity:** `OrderItemShipGroup` **Field:** `facilityId`
`lineItems[].customAttributes`	*(Triggered by match)*	`shipmentMethodTypeIdForItem = "STOREPICKUP"`	Forces the internal shipment method for this line item to be a store pickup.	**Entity:** `OrderItemShipGroup` **Field:**`shipmentMethodTypeId`
`lineItems[].customAttributes`	*(Triggered by match)*	`carrierPartyIdForItem = "_NA_"`	Removes the shipping carrier (FedEx/UPS) since it is being picked up.	**Entity:** `OrderItemShipGroup` **Field:** `carrierPartyId`
`tags`	`"bopis"`	`tags.any { it.equalsIgnoreCase(preSelectedFacTag) }`	If configured, acts as an alternate trigger for the routing rules. The tags themselves are saved for searchability.	**Entity:** `OrderAttribute` **Field:** `attrName`

`lineItems[].customAttributes`	`"key": "_pickupstore"`	`orderItemAttributes.add(...)`	Saves a flag mapping the line item to the exact Shopify property that triggered the BOPIS logic.	**Entity:** `OrderItemAttribute` **Fields:** `attrName = "StorePickupProperty", attrValue = "_pickupstore"`
`lineItems[].customAttributes`	`"key": "Delivery Method", "value": "Pick Up at Irvine..."`	`orderItemAttributes.add(...)`	All remaining custom attributes are saved as records so fulfillment staff can view the customer's notes.	**Entity:** `OrderItemAttribute` **Fields:** `attrName = "Delivery Method", attrValue = "Pick Up at Irvine..."`

Online Order (Standard Order):

Identification Logic: How the System Recognizes a Standard Order

During the Shopify import process, the script defaults to treating an order as a **standard online shipment** if it meets three strict conditions:

1. Presence of a Shipping Address (`shippingAddress`)

The JSON payload must contain a populated `shippingAddress` object. If this object is empty and the order originated from a POS system, the system bypasses standard shipping routing and treats the order as a walk-out **Cash Sale**.

2. Presence of Shipping Lines (`shippingLines`)

The payload must include a `shippingLines` array containing a shipping method title. This title represents the shipping option selected by the customer during checkout, such as **"Standard"**, **"Two-Day Delivery"**, or **"Express Shipping"**.

3. Absence of BOPIS Attributes

The script scans the `customAttributes` of every individual line item. It checks whether any attribute key contains the string **"pickupstore"** (or the equivalent dynamic property configured in OMS settings). If no pickup-related attributes are found, the items are considered eligible for standard warehouse fulfillment.

UML: How the OMS Identifies a Standard Shipping Order =>
Decision Tree of how the code works for this Online Order
(TAP THE LINK!!!)

JSON Payload Location	Extracted JSON Value	Groovy Script Logic / Variable	Effect on the OMS Order	OMS Entity &Field
shippingAddress	`{ "address1": "10645 MT SPALDING LN", "city": "ENGLEWOOD" ... }`	`boolean isShippingAddressEmpty = !shippingAddress`	Confirms the order requires physical shipping. The script maps this block to create a standard Postal Address.	**Entity:** PostalAddress **Fields:** address1, city, postalCode
sourceName	`"web"`	`def channelId = getTypeMapping(..., channelSource, ...)`	Translates `"web"` into the internal OMS Sales Channel ID (e.g., `WEB_SALES_CHANNEL`). Ensures it's not flagged as a POS Cash Sale.	**Entity:** OrderHeader **Field:** salesChannelEnumId
shippingLines[0]	`"title": "Standard"`	`shipmentMethodDesc = shippingLines[0]?.title`	Extracts the shipping speed the customer selected at checkout.	*(Used for lookup below)*
(Database Lookup)	Matches `"Standard"`	`ec.entity.find("co.hotwax.shopify.ShopifyShopCarrierShipment")`	The script queries the database for the word `"Standard"` and overrides the default method with the exact mapped equivalent.	**Entity:** OrderItemShipGroup **Field:**shipmentMethodTypeId

(Database Lookup)	Matches Carrier	`carrierPartyId = shopCarrierShipmentMethod.carrierPartyId`	Assigns the mapped carrier (e.g., `"USPS"` or `"FEDEX"`) to the order group based on the `"Standard"` lookup.	**Entity:**OrderItemShipGroup **Field:** carrierPartyId
lineItems[].customAttributes	*(No BOPIS identifiers found)*	`shipmentMethodTypeIdForItem = shipmentMethodTypeId`	Because it did not find `pickupstore` anywhere in the attributes, the item inherits the `"Standard"` shipping method and carrier from above.	**Entity:**OrderItemShipGroup **Field:**shipmentMethodTypeId
lineItems[].customAttributes	`"key": "Delivery Method", "value": "Ship to Me"`	`orderItemAttributes.add(...)`	The script still saves these standard custom attributes so the warehouse packing team can see any relevant notes (like gift wrap requests).	**Entity:**OrderItemAttribute **Fields:** attrName = `"Delivery Method"`, attrValue = `"Ship to Me"`

SendSale Order:

Send Sale / Mixed Cart POS Order Identification and Routing Logic

An order is treated as a **Send Sale** when it satisfies the following conditions:

1. The Order Originated from a POS System

The order source is mapped to:

POS_SALES_CHANNEL

This indicates that the order was created at a physical store register.

2. A Shipping Address Exists

The system checks whether a shipping address is present.

boolean isShippingAddressEmpty = !shippingAddress

If a shipping address exists, it means at least some items need to be delivered to the customer instead of being taken home immediately.

3. The Order Is Not Fully Fulfilled

The order contains items that still need fulfillment.

boolean isMixedCartPOSOrder =

```
    "POS_SALES_CHANNEL".equals(channelId) &&
    !isShippingAddressEmpty &&
    (!fulfillmentStatusText?.equals("FULFILLED"))
```

When all three conditions are true, the order is identified as a **Mixed Cart POS Order**.

Facility Determination

The system first tries to obtain the retail location from Shopify.

```
if (order.retailLocation && !tags?.any { "SENDSALE".equalsIgnoreCase(it) }) {
    locationId = order.retailLocation.legacyResourceId
}
```

If a retail location exists and the order is not explicitly tagged as `SENDSALE`, the Shopify location is captured.

For POS Cash Sale and Mixed Cart POS Orders, the system determines the facility using:

```
facilityId = resolveShopifyLocationFacility(locationId)
    ?: defaultFacilityId
    ?: "_NA_"
```

This means:

1. Try to find a facility mapped to the Shopify location.
2. If no mapping exists, use the default facility.
3. If neither is available, use `_NA_`.

Split Item Processing

Each order item is processed separately based on its fulfillment status.

Scenario 1: Item Already Fulfilled at Store

if ("FULFILLED".equalsIgnoreCase(splitType))

For fulfilled items:

effectiveShipmentMethod = "POS_COMPLETED"
effectiveCarrierPartyId = "_NA_"
effectiveFromFacilityId = facilityId

Meaning:

- The item has already been handed to the customer in-store.
- No shipping carrier is required.
- The retail store remains the fulfillment source.

Scenario 2: Item Still Unfulfilled

else if(!"STOREPICKUP".equals(effectiveShipmentMethod))

For items that still need fulfillment:

effectiveFromFacilityId = defaultFacilityId

provided no facility was pre-selected.

Meaning:

- The item is not fulfilled at the store.
- The item is not a store pickup order.
- Fulfillment is moved to the default warehouse.

This is the key behavior that makes the order function as a **Send Sale**.

How Send Sale Works in Practice

Consider the following scenario:

- Customers visit a retail store.
- The desired item is not available in that store.
- The associate creates a POS order.
- The customer provides a shipping address.
- The item remains unfulfilled.

The system identifies: POS Order + Shipping Address Present + Not Fulfilled = Mixed Cart POS Order

During split processing:

Fulfilled Items → Stay with the Store
Unfulfilled Items → Routed to Warehouse

Decision Tree: How the OMS Identifies a SendSale Shipping Order =>
Decision Tree of how the code works for the SendSale Order
(TAP THE LINK!!!)

JSON Payload Location	Extracted JSON Value	Groovy Script Logic / Variable	Effect on the OMS Order	OMS Entity & Field
sourceName	"pos"	`channelId = getTypeMapping(..., "pos", ...)`	Translates `"pos"` to the internal system ID (e.g., `POS_SALES_CHANNEL`).	**Entity:** OrderHeader **Field:** salesChannelEnumId
shippingAddress	`{ "address1": "10221 Kaimu Drive"... }`	`isShippingAddressEmpty = false`	Confirms the customer is not walking out with the item and requires physical shipping.	**Entity:** PostalAddress **Fields:** address1, city, postalCode
displayFulfillmentStatus	"UNFULFILLED"	`isMixedCartPOSOrder = true`	The combination of a POS channel + Shipping Address + Unfulfilled status flags this as a Send Sale requiring warehouse fulfillment.	*(Internal boolean used for routing)*
shippingLines[0]	`"title": "Free Expedited"`	`shipmentMethodDesc = shippingLines[0]?.title`	Extracts the shipping speed selected by the cashier. Used to query the shipment method mapping table.	**Entity:** OrderItemShipGroup **Field:** shipmentMethodTypeId

lineItems[0]	`"unfulfilledQuantity": 1`	`splitType = "UNFULFILLED"`	Confirms the individual item was not handed to the customer and must be grouped into a shipping ship group.	*(Internal string used for bucketing)*
retailLocation	`"legacyResourceId": "20523548731"`	`effectiveFromFacilityId = defaultFacilityId`	**Crucial Step:** Because this is an unfulfilled item from a POS mixed cart, the system deliberately ignores the store's ID (`20523548731`) and overrides it with the default e-commerce warehouse facility ID.	**Entity:** OrderItemShipGroup **Field:** facilityId

POS Orders:

Cash Sale Order Identification and Processing Logic

How the System Identifies a Cash Sale Order

An order is automatically identified as a **Cash Sale Order** when the following two conditions are met:

1. The Order Originated from a POS System

The order source is:

"sourceName": "pos"

which is internally mapped to:

POS_SALES_CHANNEL

This tells the OMS that the order was created at a physical store register.

2. No Shipping Address Exists

The order does not contain a shipping address.

boolean isShippingAddressEmpty = !shippingAddress

If the shipping address is missing, the system understands that the customer is not expecting a delivery.

Cash Sale Identification

The system combines both checks:

boolean isCashSaleOrder =
    isShippingAddressEmpty &&
    "POS_SALES_CHANNEL".equals(channelId) If both conditions are true, the order is identified as a Cash Sale Order.

What Happens After the Order Is Identified as a Cash Sale?

1. The Sale Is Linked to the Physical Store

The system reads the Shopify retail location and converts it into the corresponding OMS facility.

facilityId = resolveShopifyLocationFacility(locationId)

2. Shipping Is Completely Bypassed

Since the customer already has the item, no shipping process is required.

The system explicitly sets:

shipmentMethodTypeId = "POS_COMPLETED"
carrierPartyId = "_NA_"

Meaning:

- Shipment Method = POS_COMPLETED
- Carrier = Not Applicable

- No shipping label is required
- No carrier assignment is required

3. The Item Is Marked as Fulfilled

Shopify records the item as already fulfilled because the customer took possession of it at the store.

The import logic detects that:

Fulfilled Quantity = Ordered Quantity

Since everything has already been fulfilled, the item is assigned the status:

ITEM_COMPLETED

instead of:

ITEM_CREATED

4. The Order Skips Warehouse Fulfillment

Because:

- The item is already fulfilled,
- The shipment method is POS_COMPLETED,
- No carrier is assigned,

the order never enters any warehouse fulfillment process.

The warehouse team does not need to:

- Pick inventory
- Pack inventory

- Generate shipping labels
- Ship the order

The transaction is already complete.

Simple Business Flow

POS Order
+
No Shipping Address
↓
Cash Sale Order
↓
Assign Store Facility
↓
Set Shipment Method = POS_COMPLETED
↓
Remove Carrier
↓
Mark Item as ITEM_COMPLETED
↓
Skip Warehouse Fulfillment
↓
Order Complete

Final Summary

A Cash Sale Order is identified when:

- The order comes from a POS channel.
- No shipping address is provided.

Once identified, the OMS:

- Associates the sale with the retail store where it occurred.
- Sets the shipment method to POS_COMPLETED.
- Removes any shipping carrier.
- Marks the items as completed.
- Skips all warehouse fulfillment activities.

As a result, the order is considered fully completed during import because the customer has already received the purchased items at the store.

JSON Payload Location	Extracted JSON Value	Groovy Script Logic / Variable	Effect on the OMS Order	OMS Entity & Field
sourceName	`"pos"`	`channelId = getTypeMapping(..., "pos", ...)`	Translates `"pos"` into the internal system ID (e.g., `POS_SALES_CHANNEL`).	**Entity:** OrderHeader **Field:** salesChannelEnumId
shippingAddress	`null`	`isShippingAddressEmpty = true`	The absence of a shipping address confirms the customer is taking the item with them from the store (Carry-Out Sale).	*(Triggers the boolean logic below)*
sourceName + shippingAddress	`"pos"` + `null`	`isCashSaleOrder = true`	Primary identification step. A POS order without a shipping address is classified as a Cash Sale Order.	*(Internal boolean used for routing decisions)*

retailLocation	`"legacyResourceId": "87821025411"`	`facilityId = resolveShopifyLocationFacility(locationId)`	The Shopify retail location is mapped to an OMS facility so inventory is deducted from the correct physical store.	**Entity:** OrderItemShipGroup **Field:** facilityId
isCashSaleOrder	*(Triggered by match above)*	`shipmentMethodTypeId = "POS_COMPLETED"`	Overrides the default shipment method and indicates that no shipping activity is required because the customer already received the item.	**Entity:** OrderItemShipGroup **Field:**shipmentMethodTypeId
isCashSaleOrder	*(Triggered by match above)*	`carrierPartyId = "_NA_"`	Removes carrier assignment because no shipment will be generated.	**Entity:** OrderItemShipGroup **Field:** carrierPartyId
lineItems[].unfulfilledQuantity	`0`	`splitType = "FULFILLED"statusId = "ITEM_COMPLETED"`	Since the ordered quantity has already been fulfilled at the store, the item is immediately marked as completed and bypasses warehouse processing.	**Entity:** OrderItem **Field:** statusId

fulfilledQty = originalQty	Example: 1 = 1	`itemMap.statusId = "ITEM_COMPLETED"`	Confirms the entire item quantity has already been delivered to the customer and no additional fulfillment work is required.	**Entity:** OrderItem **Field:** statusId
Final OMS Outcome	POS Order + No Shipping Address + Fulfilled Item	Cash Sale Processing Flow	The order is completed during import, inventory is deducted from the retail store, shipping is skipped, and the order never enters warehouse fulfillment queues.	**Entities:** OrderHeader, OrderItem, OrderItemShipGroup

---

MixedCart Orders:

Mixed Cart Processing Logic

The system handles Mixed Cart orders using two different approaches:

1. **Special POS Mixed Cart Logic** (for orders created in physical stores)
2. **Universal Ship Group Bucketing Engine** (for all order types)

1. POS Mixed Cart Logic

A POS Mixed Cart Order is identified when:

Order Source = POS
+
Shipping Address Exists
+
Order is Not Fully Fulfilled

This is determined by:

```
boolean isMixedCartPOSOrder =
        "POS_SALES_CHANNEL".equals(channelId) &&
        !isShippingAddressEmpty &&
        (!fulfillmentStatusText?.equals("FULFILLED"))
```

Why is this needed?

In a store transaction, some items may be handed to the customer immediately, while other items may need to be shipped later.

Example:

Item	Outcome
Necklace	Customer takes home
Bracelet	Ship from warehouse

The system must separate these items and route them differently.

POS Mixed Cart Routing Logic

When the order is identified as a Mixed Cart POS Order:

Fulfilled Items

if ("FULFILLED".equalsIgnoreCase(splitType))

The item was already handed to the customer.

The system sets:

effectiveShipmentMethod = "POS_COMPLETED"
effectiveCarrierPartyId = "_NA_"
effectiveFromFacilityId = facilityId

Result:

- No shipping required.
- No carrier required.
- Inventory is deducted from the retail store.

Unfulfilled Items

else if (!"STOREPICKUP".equals(effectiveShipmentMethod))

The item still needs fulfillment.

The system routes it to:

effectiveFromFacilityId = defaultFacilityId

Result:

- Inventory comes from the warehouse.
- Item enters warehouse fulfillment.
- Warehouse picks, packs, and ships the item.

This is effectively how a Send Sale is processed.

2. Universal Ship Group Bucketing Engine

For all order types, including:

- Web Orders
- BOPIS Orders
- Ship-to-Home Orders
- Mixed Cart Orders

the system processes each item individually.

lineItemsRaw.each { lineItem ->

Each item is evaluated separately.

Building the Ship Group Key

For every item, the system creates a routing key:

```
String shipGroupKey =
    "${bucketFacilityId}|${effectiveShipmentMethod}|${effectiveCarrierPartyId}|${splitType}"
```

The key contains:

Component	Purpose
bucketFacilityId	Where inventory comes from
effectiveShipmentMethod	Shipping method
effectiveCarrierPartyId	Carrier

splitType	FULFILLED or UNFULFILLED

Bucket Creation

The system checks:

if (!shipGroupBuckets.containsKey(shipGroupKey))

If the bucket does not exist, it creates one.

Then:

shipGroupBuckets[shipGroupKey].add(splitItem)

The item is added to that bucket.

Items with identical keys go into the same ship group.

Items with different keys create separate ship groups.

Example: Web Mixed Cart

Customer purchases:

Item	Delivery Type
T-Shirt	Ship to Home
Hat	Store Pickup

T-Shirt

Generated key:

WAREHOUSE_1|STANDARD|FEDEX|UNFULFILLED

Hat

Generated key:

STORE_29|STOREPICKUP|_NA_|UNFULFILLED

Since the keys are different:

Bucket 1
WAREHOUSE_1|STANDARD|FEDEX|UNFULFILLED
└── T-Shirt

Bucket 2
STORE_29|STOREPICKUP|_NA_|UNFULFILLED
└── Hat

The OMS automatically creates two different ship groups.

Final Summary

POS Mixed Cart Orders

The system uses special logic to separate:

Fulfilled Items
↓
POS_COMPLETED
↓
Stay with Store

Unfulfilled Items
↓
Route to Warehouse
↓
Send Sale Fulfillment

All Other Mixed Carts

The system relies on the ship group bucketing engine:

Item
↓
Generate Ship Group Key
↓
Find/Create Bucket
↓
Place Item Into Bucket
↓

Create Ship Group

Any items with different fulfillment requirements automatically end up in separate ship groups.

Decision Tree: How the OMS Identifies a MixedCart Shipping Order =>
Decision Tree of how the code works for the MixedCart Order
(TAP THE LINK!!!)

Identifier / Logic Variable	Groovy Script Code	Effect on the OMS Order	OMS Entity & Field
bucketFacilityId	*(e.g., "WAREHOUSE_1" vs "STORE_29")*	Determines which physical facility is responsible for fulfilling the item (warehouse or store).	**Entity:** OrderItemShipGroup **Field:** facilityId
effectiveShipmentMethod	*(e.g., "STANDARD" vs "STOREPICKUP")*	Determines how the item reaches the customer (SendSale, Store Pickup, POS Completed, etc.).	**Entity:** OrderItemShipGroup **Field:** shipmentMethodTypeId
effectiveCarrierPartyId	*(e.g., "FEDEX" vs "_NA_")*	Determines which shipping carrier will be used for delivery when shipping is required.	**Entity:** OrderItemShipGroup **Field:** carrierPartyId

shipGroupBuckets	`shipGroupBuckets[shipGroupKey].add(...)`	**Final Grouping Logic:** For every unique combination of Facility + Shipment Method + Carrier + Split Type, the OMS creates a separate Ship Group.	**Entity:** OrderItemShipGroup*(Creates a new Ship Group record for each unique bucket)*
splitItem	`shipGroupBuckets[shipGroupKey].add(splitItem)`	Associates an individual Order Item with its corresponding Ship Group. This establishes which fulfillment group the item belongs to.	**Entity:**OrderItemShipGroup **Fields:** orderItemSeqId, shipGroupSeqId